ISTA 130 Midterm Study Guide

Key skills:

- **Define a function.**
- **Call a function and appropriately use the returned value.**
- **`for` and `while` loops:**
  - **Traverse by index.**
  - **Build a string.**
  - **Accumulate a sum.**
  - **Convert between if possible.**
- **Create and evaluate arithmetic, Boolean, and string expressions, and combinations thereof.**
- **Create conditional statements, i.e. `if-elif-else`.**
- **`print` and `input`.**
- **Index a string.**

Key resources:

- **Worksheets and solutions, plus other example code.**
- **PowerPoint PDF's.**
- **Discussion section PDF's, exercises, and solutions.**
- **`All_about_functions.pdf`.**

The test will be 70% multiple choice, 30% code writing.

Know how to document a function – what the function does, parameter name, types, and meanings, and return types and meanings.

Know how to evaluate arithmetic expressions. These expressions use the arithmetic operators +, -, `*`, `/`, `//`, `%`, `**`. Understand the order of evaluation (PEMDAS). PEMDAS stands for Parentheses, Exponentiation, Multiplication, Division, Addition, and Subtraction. This means that anything in parentheses gets evaluated first and exponentiation second. Multiplication, division, integer division, and modulus (`*`, `/`, `//`, `%`) all have the same third level precedence. Addition and subtraction are last. Operators with the same precedence are evaluated left to right except exponentiation, which goes right to left. Understand the difference between integer and floating point division and that floating point division always returns a float, even if the dividend and divisor were integers. How does Python represent a whole number that is a float (one that has no fractional part, like 7, 22, or -13, as opposed to 5.4 and -3.333)? Example arithmetic expression:

```
(num_squares + num_circles) / num_polygons
```

Know how to evaluate Boolean expressions, which always evaluate to either `True` or `False`. These expressions often use the comparison operators <, >, <=, >=, ==, !=. They can also contain arithmetic expressions and operators and string expressions and operators. Example:

```
(num_squares + num_circles) / num_polygons <= 40
```

Notice that the above Boolean expression doesn't contain the keyword `if` anywhere. `if` statements make use of Boolean expressions, however Boolean expressions can exist without having an associated `if` statement. If you are asked to write a Boolean expression, do not include `if`.

Understand the logical operators and their precedence: `not`, `and`, and `or`. These are used to create compound Boolean expressions.

Know what happens when a value is used as a condition. In other words, know what values of `x` will make the body of this `if` statement execute and what values won't if `x` is a variable referencing an integer, a float, or a string:

```
if x:
    <do stuff>
```

Be able to evaluate this expression:

`k / 3 - k // 3`, where k is an integer.

Understand the following terms: *function signature, function header (same as signature), function definition, function body, function call, variable, literal, parameter, argument, arithmetic expression, condition, statement, conditional statement, compound statement, scope, loop control variable, Boolean expression, assignment statement, directory, path, index, augmented assignment, immutable, traverse by index*.

Know how to create and evaluate string expressions using concatenation, repetition, and indexing. Know the `len()` function for strings. Know that string literals are always enclosed in quotes, with the exception of user input (see the `input()` section below). Make sure you know how to distinguish between a variable name and a string literal. Know what the empty string is and its literal in Python. Know how to do string repetition and string concatenation. Know how to build a string using a loop.

Know that `input()` always returns a string, although the user doesn't type quotes. Be very aware that `input` takes only zero or one arguments.

Know your escape characters, at least `'\n'` and `'\t'`. Know how to include quotes in a string without giving Python a headache (there are two ways).

Know the `int`, `float`, and `str` typecasting functions and how to use them.

Understand how the `print()` function works. Know that it can take multiple arguments (separated by commas) and that these arguments are printed out separated by single spaces. Understand what happens when you print a string variable, concatenated string variables, and multiplied string variables.

For both `input` and `print`, watch out for missing parentheses!

Know the difference between `print` and `return`.

Know how to save the value a function returns when called into a variable.

Know how to incorporate the `random` module into your code and to use `random.randint`.

Understand conditional execution of code – in other words `if – elif – else` statements. Be able to write code containing such statements. As part of this, you will need to understand Python indentation. You will also have to be able to create Boolean expressions in Python that define conditions for execution. Know that `else` catches all remaining cases and therefore does not need a condition.

`turtle` will not be on the exam.

Be able to translate instructions given in English into Python.

Be able to define functions given specifications about their functionality and parameters. As part of this, you will have to understand Python indentation and know the function definition syntax:

```
def fname(param1, param2, …):
     <function body>
```

Make sure you know how to call functions and to use the argument-parameter passing process. Know what can be passed as an argument and what can be used to define a parameter. Understand the benefits of functions.

Understand scope. This means that a variable created in one function can't be used in another.

Understand and be able to write `for` and `while` loops. `for` loop example:

```
for i in range(7):
    print((i + 1)**2)
```

Know how to use a loop to update a variable, i.e. accumulate a sum or build a string. To do this, you must initialize a variable before the loop and update it inside the loop. For example:

```
word = 'dog'
weird = ''
for i in range(len(word)):
    weird += word[i] * 3
```

What is `weird`?

You should be able to look at the above loop and know what it does. You should be able to show what it does using an execution trace.

Know what a string variable indexed by an integer evaluates to. For example:

```
string = 'Elephant'
print(string[2])
```

What does the above code snippet print?

Know what `for` and `while` loop control variables are, how they work, and what you can do with them. Know how to traverse by index, accessing the elements of sequence in order.

Know how to convert `for` loops to `while` loops and vice versa.

Know when to use `while True` and how to end such a loop.

You do NOT need to know:

- **Traverse by element.**
- **Slicing.**
- **String methods.**
- **Nested loops.**